PART 2

Resilience & Messaging

9 Service Discovery
10 Circuit Breakers
11 Timeouts
12 Retries
13 Exponential Backoff
14 Idempotency
15 Message Queues
16 Pub/Sub
17 Event-Driven Architec…
18 Distributed Transactio…
19 Saga Pattern
20 Dead Letter Queues

#009

Service Discovery

WHAT IT IS

How services find each other when instances come and go and IPs change.

WHY IT MATTERS

Hard-coded IPs break the moment you scale or redeploy.

EXAMPLE

```
Http::baseUrl(config('services.billing.url'))->get(
    '/invoices'); // url comes from env, resolved
    via DNS
```

#010

Circuit Breakers

WHAT IT IS

After repeated failures calling a dependency, "open" the circuit and fail fast instead of piling up requests. Later, allow a trial request ("half-open").

WHY IT MATTERS

Prevents cascading failures and gives the broken service room to recover.

EXAMPLE

```
function callWithBreaker(string $key, Closure $call,
    Closure $fallback) {
    if (Cache::get("cb:$key:open")) return $fallback();
    try {
        $r = $call();
        Cache::forget("cb:$key:fails");
        return $r;
    } catch (Throwable $e) {
        $fails = Cache::increment("cb:$key:fails");
        if ($fails >= 5) Cache::put("cb:$key:open", true,
    30); // open 30s
        return $fallback();
…
```

#011

Timeouts

WHAT IT IS

A maximum time you will wait for an operation.

WHY IT MATTERS

Without timeouts, one slow dependency ties up every worker and the whole app stalls.

EXAMPLE

```
Http::connectTimeout(2)->timeout(5)->get(
    'https://api.partner.com/rates');
```

#012

Retries

WHAT IT IS

Re-attempting an operation that failed for transient reasons (network blips, 503s).

WHY IT MATTERS

Many failures are temporary; retries hide them from users. Only retry safe or idempotent operations.

EXAMPLE

```
Http::retry(3, 200, throw: false)->post($url,
    $payload);

class SendInvoice implements ShouldQueue {
    public int $tries = 5;
}
```

#013

Exponential Backoff

WHAT IT IS

Increasing the wait between retries (1s, 2s, 4s, 8s...), usually with jitter (random offset).

WHY IT MATTERS

Prevents a "retry storm" where all clients hammer a recovering service simultaneously.

EXAMPLE

```
class SyncToCrm implements ShouldQueue {
    public int $tries = 6;
    public function backoff(): array { return [1, 5,
    30, 120, 600]; }
}
// With jitter:
$delay = min(300, (2 ** $attempt) + random_int(0,
    1000) / 1000);
```

#014

Idempotency

WHAT IT IS

Doing an operation many times has the same effect as doing it once.

WHY IT MATTERS

Retries, double-clicks and duplicate webhooks must not charge a customer twice.

EXAMPLE

```
public function charge(Request $r) {
    $key = $r->header('Idempotency-Key') ?? abort(422);
    return Cache::lock("idem:$key", 10)->block(5,
    function () use ($key, $r) {
        if ($saved = Cache::get("idem:result:$key"))
    return $saved;
        $result = $this->stripe->charge($r->validated(
    ));
        Cache::put("idem:result:$key", $result, now(
    )->addDay());
        return $result;
    });
}
```

#015

Message Queues

WHAT IT IS

Work is placed on a queue and processed asynchronously by workers.

WHY IT MATTERS

Fast HTTP responses, smoothing traffic spikes, isolating slow tasks (emails, PDFs, imports).

EXAMPLE

```
SendWeeklyReport::dispatch($user)->onQueue('emails');
// php artisan queue:work redis --queue=emails,
    default --tries=3
```

#016

Pub/Sub

WHAT IT IS

Publishers emit messages to a topic; any number of subscribers receive them, with no direct coupling.

WHY IT MATTERS

Many consumers react to one event independently.

EXAMPLE

```
Redis::publish('task.completed', json_encode(['id'
    => $task->id]));

// subscriber (artisan command)
Redis::subscribe(['task.completed'], function (
    $message) {
    // update analytics, notify Slack...
});
```

#017

Event-Driven Architecture

WHAT IT IS

Components communicate by emitting and reacting to events ("TaskCompleted") instead of calling each other directly.

WHY IT MATTERS

Loose coupling; new behaviour is added by adding listeners.

EXAMPLE

```
event(new TaskCompleted($task));

class TaskCompleted { public function __construct(
    public Task $task) {} }

// Listeners: NotifyAssignee, UpdateBurndown,
    AwardPoints (each ShouldQueue)
```

#018

Distributed Transactions

WHAT IT IS

One logical operation that spans multiple databases or services and must succeed or fail as a whole.

WHY IT MATTERS

A local DB::transaction cannot cover "charge Stripe + create subscription + email user".

EXAMPLE

```
DB::transaction(function () use ($order) {
    $order->update(['status' => 'paid']);
    OutboxMessage::create(['type' => 'order.paid',
    'payload' => $order->toArray()]); // published
    later by a worker
});
```

#019

Saga Pattern

WHAT IT IS

A long operation is split into local steps, each with a compensating action to undo it on failure.

WHY IT MATTERS

Gives you "all or nothing" semantics across services without global locks.

EXAMPLE

```
Bus::chain([
    new ReserveStock($order),
    new ChargeCard($order),
    new CreateShipment($order),
])->catch(fn (Throwable $e) => dispatch(new
    CompensateOrder($order)))->dispatch();
```

#020

Dead Letter Queues

WHAT IT IS

A holding place for messages/jobs that failed too many times.

WHY IT MATTERS

Poison messages do not block the queue, and nothing is silently lost; you can inspect and replay.

EXAMPLE

```
php artisan queue:failed          # list
php artisan queue:retry <uuid>    # replay one
php artisan queue:retry all
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).